

HANDS-ON TUGAS BESAR

Implementasi Aplikasi Chat Real-Time Berbasis Cloud Menggunakan Docker dan AWS

Studi Kasus: Backend BECHAT dan Frontend FECHAT

Aspek	Deskripsi
Jenis Dokumen	Dokumentasi hands-on implementasi Tugas Besar
Aplikasi	Chat real-time cloud-native BECHAT dan FECHAT
Backend	FastAPI, Socket.IO, SQLAlchemy Async, JWT, bcrypt
Frontend	React 19, Vite, TypeScript, Tailwind CSS, React Router
Database	SQLite development dan PostgreSQL 15 container
Container	Docker multi-stage, Docker Compose, dan Nginx
Cloud	Amazon ECR/Docker Hub dan deployment AWS EC2/ECS
Repository	BECHAT: github.com/diffarydk/BECHAT FECHAT: github.com/diffarydk/chatFE
Tahun	2026

CLOUD COMPUTING

IMPLEMENTASI FULL-STACK DAN DEPLOYMENT CLOUD

PENDAHULUAN

Dokumen ini menjelaskan proses implementasi aplikasi chat real-time secara full-stack. Bagian I membahas backend BECHAT dan melanjutkan hasil hands-on backend yang telah diuji pada 5 Juni 2026. Bagian II membahas frontend FECHAT dan cara menghubungkannya ke kontrak REST API serta Socket.IO backend.

BECHAT menangani autentikasi, database, REST API, file sharing, dan event real-time. FECHAT menjadi client React yang menyimpan session, melindungi route privat, memanggil API, menampilkan workspace chat, dan dapat diaktifkan untuk menerima pesan Socket.IO tanpa refresh.

Teknologi yang Digunakan

Lapisan	Teknologi	Fungsi
Backend API	FastAPI + Uvicorn	REST API berbasis ASGI
Real-time	python-socketio	Event pesan dan status pengguna
Database	SQLAlchemy Async	ORM SQLite/PostgreSQL
Security	JWT + bcrypt	Token dan hashing password
Frontend	React + Vite	Single-page application
UI	Tailwind CSS + Motion	Layout dan animasi
Container	Docker + Nginx	Runtime konsisten
Cloud	AWS + registry image	Distribusi dan deployment

Alur Implementasi Full-Stack

```

Frontend FECHAT :3000 / :80
  |
  | REST API + Bearer JWT
  | Socket.IO + auth token
  v
Backend BECHAT :5000
  |
  +-> PostgreSQL :5432
  +-> uploads / Amazon S3

GitHub -> CI/CD -> Image Registry -> AWS EC2/ECS

```

BAGIAN I - HANDS-ON BACKEND BECHAT

1. PERSIAPAN ENVIRONMENT

Implementasi backend dilakukan menggunakan Python, Docker Desktop, Docker Compose, dan Git. Pastikan seluruh aplikasi pendukung tersedia.

1. Periksa versi Python, Git, Docker, dan Docker Compose.
2. Masuk ke repository backend D:\BECHAT.
3. Periksa branch aktif dan remote repository sebelum mengubah kode.

Verifikasi aplikasi pendukung

```
python --version
git --version
docker --version
docker compose version
```

Masuk ke repository

```
cd D:\BECHAT
git remote -v
git branch --show-current
```

2. STRUKTUR PROJECT BECHAT

Repository backend disusun modular agar konfigurasi, model, schema, autentikasi, endpoint, dan data seed tidak berada dalam satu file.

```
BECHAT/
|-- .github/workflows/docker-be.yml
|-- app/
|   |-- main.py
|   |-- config.py
|   |-- database.py
|   |-- models.py
|   |-- schemas.py
|   |-- auth.py
|   |-- seed.py
|-- Dockerfile
|-- docker-compose.yml
|-- requirements.txt
|-- .env.example
`-- BACKEND_INTEGRATION.md
```

File	Isi Implementasi
app/main.py	FastAPI, REST endpoint, Socket.IO, lifecycle
app/config.py	Environment variable dan Pydantic Settings
app/database.py	Async engine dan session SQLAlchemy
app/models.py	Struktur tabel dan relationship
app/schemas.py	Validasi request dan response
app/auth.py	bcrypt, JWT, dan HTTPBearer
app/seed.py	Data awal untuk pengujian

3. ARSITEKTUR BACKEND DAN CLOUD

Backend menggabungkan FastAPI dan Socket.IO pada satu ASGI application agar REST API dan koneksi real-time memakai port 5000 yang sama.

```

Browser / Frontend FECHAT
|
| REST API + Bearer JWT
| Socket.IO + auth token
v
FastAPI + Socket.IO ASGI :5000
|
+-> SQLAlchemy Async -> PostgreSQL :5432
+-> uploads/ volume -> file sharing

```

Pembuatan ASGI application

```

app = FastAPI(title=settings.APP_NAME, lifespan=lifespan)

sio = socketio.AsyncServer(async_mode="asgi")
sio_asgi_app = socketio.ASGIApp(
    socketio_server=sio,
    other_asgi_app=app,
    socketio_path="/socket.io",
)

```

4. KONFIGURASI ENVIRONMENT

.env.example

```

JWT_SECRET=your_jwt_secret_key_here
DATABASE_URL=postgresql+psycopg://username:password@host:port/database
CORS_ORIGINS=["http://localhost:3000", "http://localhost:5173"]

```

Development dengan SQLite

```

JWT_SECRET=ganti-dengan-random-secret-yang-panjang
DATABASE_URL=sqlite+aiosqlite:///./app.db
CORS_ORIGINS=["http://localhost:3000"]

```

Perhatian credential

Jangan menyimpan JWT secret, password database, access key AWS, atau private key .pem di repository Git.

5. IMPLEMENTASI STRUKTUR DATABASE

Tabel	Kolom Utama	Kegunaan
users	id, name, email, password_hash, status	Akun/presence
chats	id, name, type, last_message	Direct/group chat
chat_members	chat_id, user_id, role	Relasi user-chat
messages	chat_id, sender_id, content, status	Riwayat pesan
friend_requests	from_id, to_id, status	Pertemanan
files	owner_id, chat_id, name, url	Metadata file

Contoh model Message

```
class Message(Base):
    __tablename__ = "messages"
    id = Column(String, primary_key=True)
    chat_id = Column(String, ForeignKey("chats.id"))
    sender_id = Column(String, ForeignKey("users.id"))
    content = Column(String, nullable=False)
    status = Column(String, default="sent")
```

6. IMPLEMENTASI AUTENTIKASI DAN SECURITY

Password disimpan sebagai hash bcrypt. Setelah login berhasil, backend membuat JWT yang dikirim frontend melalui Authorization Bearer.

Hashing password

```
def hash_password(password: str) -> str:
    password_bytes = password.encode("utf-8")
    return bcrypt.hashpw(password_bytes, bcrypt.gensalt()).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))
```

4. Frontend mengirim email dan password ke POST /api/auth/login.
5. Backend mencari user dan memverifikasi password dengan bcrypt.
6. Backend membuat JWT jika credential benar.
7. Frontend menyimpan token dan mengirim Bearer token pada API privat.

7. IMPLEMENTASI REST API

Method	Endpoint	Fungsi
POST	/api/auth/register	Membuat akun
POST	/api/auth/login	Login dan JWT
GET	/api/chats	Daftar chat
GET	/api/chats/{id}/messages	Riwayat pesan
GET	/api/members	Direktori member
GET	/api/friend-requests	Request pending
POST	/api/friend-requests	Mengirim request
POST	/api/friend-requests/{id}/accept	Menerima request
POST	/api/friend-requests/{id}/reject	Menolak request
GET/POST	/api/files	Daftar dan upload file

Contoh request login

```
POST http://localhost:5000/api/auth/login
Content-Type: application/json

{
  "email": "ada@example.com",
  "password": "Secret123"
}
```

8. IMPLEMENTASI CHAT REAL-TIME DENGAN SOCKET.IO

Integrasi client

```
const socket = io('http://localhost:5000', {
  auth: { token: localStorage.getItem('token') }
});

socket.emit('joinChat', { chatId: 'c1' });
socket.emit('sendMessage', {
  chatId: 'c1',
  content: 'Halo dari frontend'
});
```

Arah	Event	Proses
Client -> Server	joinChat	Masuk room chat
Client -> Server	leaveChat	Keluar room
Client -> Server	sendMessage	Simpan/broadcast pesan
Server -> Client	receiveMessage	Pesan baru
Server -> Client	chatUpdated	Update sidebar chat
Server -> Client	memberStatusUpdate	Presence
Server -> Client	friendRequestReceived	Notifikasi request

9. IMPLEMENTASI FRIEND REQUEST DAN FILE SHARING

Ketika friend request diterima, backend membuat direct chat dan menambahkan kedua user sebagai anggota. File diunggah memakai multipart/form-data; binary disimpan pada volume dan metadata pada DB.

Penyimpanan production

Local volume sesuai untuk satu instance. Deployment multi-instance sebaiknya menggunakan Amazon S3 dan signed URL.

10. DATA SEEDING

User ID	Nama	Email	Password Uji	Role
ada-77	Ada Lovelace	ada@example.com	Secret123	Admin
bob-99	Bob Engineer	bob@example.com	Secret123	Engineer
elena	Elena Rostova	elena@example.com	Secret123	Admin
marcus	Marcus Chen	marcus@example.com	Secret123	Engineer

Khusus development

Credential seed bersifat publik dan harus diganti atau dinonaktifkan pada production.

11. MENJALANKAN BACKEND SECARA LOKAL

Mode SQLite

```
cd D:\BECHAT
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m uvicorn app.main:sio_asgi_app --reload --port 5000
```

Layanan	Alamat
Root/health	http://localhost:5000/
Swagger UI	http://localhost:5000/docs
OpenAPI JSON	http://localhost:5000/openapi.json
Socket.IO	http://localhost:5000/socket.io

12. IMPLEMENTASI DOCKER

Ringkasan Dockerfile

```
FROM python:3.13-slim as builder
WORKDIR /app
RUN python -m venv /opt/venv
COPY requirements.txt .
RUN /opt/venv/bin/pip install -r requirements.txt

FROM python:3.13-slim as runner
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY app /app/app
EXPOSE 5000
CMD ["uvicorn", "app.main:sio_asgi_app", "--host", "0.0.0.0", "--port", "5000"]
```

Menjalankan container

```
docker compose up --build -d
docker compose ps
docker compose logs -f backend
```

13. HASIL PENGUJIAN DOCKER DAN DATABASE

Pengujian backend menunjukkan container backend dan PostgreSQL healthy, seed berhasil, dan enam tabel utama terbentuk.

Verifikasi tabel PostgreSQL

```
public | chat_members | table
public | chats         | table
public | files         | table
public | friend_requests | table
public | messages      | table
public | users         | table
```

14. HASIL PENGUJIAN REST API

Status pengujian

POST /api/auth/login, GET /api/chats, GET /api/members, dan GET /api/friend-requests menghasilkan HTTP 200.

15. HASIL PENGUJIAN SOCKET.IO**Hasil**

Client terhubung menggunakan JWT, bergabung ke chat c1, dan menerima event receiveMessage tanpa refresh halaman.

16. VERSION CONTROL DAN GITHUB

```
git status
git add app Dockerfile docker-compose.yml requirements.txt
git commit -m "feat: implement real-time cloud chat backend"
git push origin main
```

File yang tidak boleh di-push

```
.env
.venv/
__pycache__/
*.db
uploads/
*.pem
```

17. GITHUB ACTIONS DAN AMAZON ECR

8. Checkout source code.
9. Menyiapkan Docker Buildx.
10. Login ke Amazon ECR menggunakan credential pipeline.
11. Build image backend.
12. Push tag latest dan tag berdasarkan Git commit SHA.

Rekomendasi

Gunakan GitHub OpenID Connect dan IAM Role untuk menghindari long-lived access key.

18. DEPLOYMENT BACKEND KE AWS

Port	Source	Kegunaan
22	IP administrator	SSH
80	0.0.0.0/0	HTTP reverse proxy
443	0.0.0.0/0	HTTPS dan WSS
5000	Terbatas/internal	Debug backend

Install Docker pada EC2

```
sudo apt update
sudo apt install docker.io docker-compose-plugin awscli -y
sudo usermod -aG docker $USER
newgrp docker
```

Login dan pull image

```
aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin \
058264530865.dkr.ecr.us-east-1.amazonaws.com

docker pull 058264530865.dkr.ecr.us-east-1.amazonaws.com/fechat-be:latest
```

19. DOMAIN, HTTPS, DAN WEBSOCKET

Production membutuhkan HTTPS untuk REST API dan WSS untuk Socket.IO. Reverse proxy atau load balancer harus meneruskan upgrade WebSocket.

Contoh konfigurasi Nginx

```
server {
    listen 443 ssl;
    server_name api.example.com;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

20. SECURITY HARDENING

- Ganti JWT_SECRET dan simpan pada secret manager.
- Batasi CORS hanya untuk domain frontend yang sah.
- Jangan expose PostgreSQL ke internet.
- Gunakan RDS private subnet dan migration Alembic.
- Pindahkan file upload ke S3 serta validasi ukuran dan MIME.
- Tambahkan rate limiting login/register.
- Gunakan image tag SHA dan aktifkan monitoring.
- Rotasi credential atau private key yang pernah tersimpan di Git.

21. TROUBLESHOOTING BACKEND

Masalah	Penyebab dan Penyelesaian
Docker engine tidak ditemukan	Jalankan Docker Desktop.
Port 5000 digunakan	Hentikan proses lama atau ubah port.
PostgreSQL refused	Gunakan hostname postgres di Compose.
401 Unauthorized	Kirim Bearer token yang valid.
Socket rejected	Pastikan auth.token dan JWT valid.

Seed tidak berjalan

Seed dilewati jika users sudah terisi.

WebSocket gagal HTTPS

Teruskan header Upgrade.

22. RINGKASAN IMPLEMENTASI BACKEND

Backend BECHAT telah menyediakan REST API, autentikasi JWT, database asynchronous, Socket.IO, Docker Compose, dan distribusi image. Bagian berikut melanjutkan hands-on dengan implementasi frontend FECHAT.

Titik Lanjut

Pastikan backend dapat diakses pada <http://localhost:5000> dan CORS mengizinkan <http://localhost:3000> sebelum memulai Bagian II.

BAGIAN II - HANDS-ON FRONTEND FECHAT

Bagian ini melanjutkan hands-on backend dengan repository FECHAT pada D:\FECHAT. Seluruh contoh mengikuti source code aktual. Fitur login, register, route guard, API wrapper, initial chat data, user lookup, dan friend request telah memiliki titik integrasi backend. Socket.IO dan beberapa halaman lain masih perlu diaktifkan dari implementasi mock.

23. PERSIAPAN ENVIRONMENT FRONTEND

13. Pastikan Node.js 20 atau versi LTS tersedia.
14. Masuk ke repository D:\FECHAT.
15. Install dependency dari package-lock.json.
16. Buat file .env.local untuk alamat backend.
17. Jalankan Vite pada port 3000.

Menjalankan FECHAT

```
node --version
npm --version

cd D:\FECHAT
npm install
npm run dev
```

Alamat development

Script dev menjalankan Vite pada `http://localhost:3000` dan dapat diakses dari host lain karena memakai `--host=0.0.0.0`.

24. STRUKTUR PROJECT FECHAT

```
FECHAT/
|-- src/
|   |-- components/
|   |   |-- ProtectedRoute.tsx
|   |   |-- PublicRoute.tsx
|   |-- context/AuthContext.tsx
|   |-- pages/
|   |   |-- Login.tsx
|   |   |-- Register.tsx
|   |   |-- Chat.tsx
|   |   |-- GroupChats.tsx
|   |   |-- Files.tsx
|   |   |-- Members.tsx
|   |-- services/api.ts
|   |-- App.tsx
|   |-- main.tsx
|   |-- index.css
|-- Dockerfile
|-- nginx.conf
|-- vite.config.ts
|-- package.json
|-- .env.example
```

File	Peran
src/App.tsx	Router dan daftar route aplikasi

src/context/AuthContext.tsx	State user/token dan localStorage
src/services/api.ts	Wrapper fetch dan Bearer token
src/pages/Login.tsx	Login real atau fallback mock
src/pages/Register.tsx	Register real atau fallback mock
src/pages/Chat.tsx	Chat, member, request, integrasi API
Dockerfile	Build Vite lalu serve melalui Nginx
nginx.conf	SPA fallback ke index.html

25. KONFIGURASI ENVIRONMENT FECHAT

.env.local

```
VITE_API_URL=http://localhost:5000/api
VITE_WS_URL=http://localhost:5000
```

VITE_API_URL wajib diisi untuk memakai backend nyata. Halaman Login, Register, dan Chat memeriksa environment ini; jika kosong, aplikasi menggunakan mock data untuk demonstrasi offline.

Perilaku Vite

Environment variable berawalan VITE_ dimasukkan ke bundle saat proses build. Mengubah URL pada server setelah image dibangun tidak mengubah bundle; image harus dibangun ulang.

26. ROUTING DAN ROUTE GUARD

Struktur App.tsx

```
<AuthProvider>
  <BrowserRouter>
    <Routes>
      <Route path="/login" element={<PublicRoute><Login /></PublicRoute>} />
      <Route path="/register" element={<PublicRoute><Register /></PublicRoute>} />
      <Route path="/chat" element={<ProtectedRoute><Chat /></ProtectedRoute>} />
      <Route path="/groups" element={<ProtectedRoute><GroupChats /></ProtectedRoute>} />
      <Route path="/files" element={<ProtectedRoute><Files /></ProtectedRoute>} />
      <Route path="/members" element={<ProtectedRoute><Members /></ProtectedRoute>} />
    </Routes>
  </BrowserRouter>
</AuthProvider>
```

Route	Jenis	Perilaku
/login	Publik	Redirect ke /chat jika sudah login
/register	Publik	Redirect ke /chat jika sudah login
/chat	Privat	Redirect ke /login tanpa token
/groups	Privat	Halaman group chat
/files	Privat	Halaman file
/members	Privat	Direktori member

27. AUTHCONTEXT DAN PENYIMPANAN SESSION

AuthProvider membaca token dan user dari localStorage ketika aplikasi dimulai. Route guard menunggu isLoading selesai sebelum memutuskan redirect.

Session frontend

```
const login = (newToken: string, newUser: User) => {
  setToken(newToken);
  setUser(newUser);
  localStorage.setItem('token', newToken);
  localStorage.setItem('user', JSON.stringify(newUser));
};

const logout = () => {
  setToken(null);
  setUser(null);
  localStorage.removeItem('token');
  localStorage.removeItem('user');
};
```

Catatan security

localStorage sederhana untuk hands-on, tetapi rentan jika aplikasi memiliki celah XSS. Production dapat memakai access token singkat dan refresh token pada cookie httpOnly.

28. API WRAPPER DAN BEARER TOKEN

src/services/api.ts

```
const BASE_URL = import.meta.env.VITE_API_URL ||
  'http://localhost:5000/api';

const token = localStorage.getItem('token');
if (token) {
  headers.set('Authorization', `Bearer ${token}`);
}

const response = await fetch(`${BASE_URL}${endpoint}`, config);
```

18. Wrapper mengambil token dari localStorage.
19. Authorization Bearer ditambahkan otomatis.
20. Content-Type JSON ditambahkan kecuali body berupa FormData.
21. Response non-2xx diubah menjadi Error.
22. Status 401 menghapus session dan redirect ke /login.

Kontrak response

Wrapper selalu memanggil response.json(). Endpoint sukses sebaiknya mengembalikan JSON. Download file/blob memerlukan fetch khusus.

29. INTEGRASI LOGIN DAN REGISTER

Login.tsx

```
const data = await api.post('/auth/login', {
  email,
  password
});
login(data.token, data.user);
navigate('/chat');
```

Register.tsx

```
const data = await api.post('/auth/register', {
  userId,
  name,
  email,
  password
});
login(data.token, data.user);
navigate('/chat');
```

Endpoint	Request	Response minimum
POST /auth/login	email, password	token, user{id,name,email,avatar}
POST /auth/register	userId, name, email, password	token, user{id,name,email,avatar}

Pemetaan ID

Register mengirim userId, tetapi frontend menyimpan user.id. Backend harus mengembalikan field id pada object user.

30. MEMUAT CHAT, MEMBER, DAN FRIEND REQUEST**Initial data pada Chat.tsx**

```
const chatsData = await api.get('/chats');
setChats(chatsData);

const membersData = await api.get('/members');
setMembers(membersData);

const requestData = await api.get('/friend-requests');
setFriendRequests(requestData);
```

Endpoint	Dipakai untuk
GET /chats	Sidebar direct dan group chat
GET /members	Panel member workspace
GET /friend-requests	Modal friend request
GET /users/{id}	Validasi user saat memulai chat
POST /friend-requests	Mengirim friend request

Mencari user dan mengirim friend request

```
const user = await api.get(`/users/${targetId}`);
await api.post('/friend-requests', { to: targetId });
alert(`Friend request sent to ${user.name}`);
```

31. MENGAKTIFKAN MESSAGE HISTORY

Source saat ini belum mengambil riwayat pesan ketika activeChat berubah. Tambahkan effect berikut setelah backend menyediakan endpoint GET /api/chats/{chatId}/messages.

Tambahan pada Chat.tsx

```
useEffect(() => {
  if (!activeChat || !import.meta.env.VITE_API_URL) return;

  api.get(`/chats/${activeChat}/messages`)
    .then(setMessages)
    .catch((error) => console.error(error));
}, [activeChat]);
```

Format response

Jika backend mengembalikan object pagination {items,nextCursor}, ubah setMessages(data) menjadi setMessages(data.items).

32. MENGAKTIFKAN SOCKET.IO PADA FECHAT

Chat.tsx saat ini hanya berisi placeholder Socket.IO dan simulasi status sent/delivered/read. Install client lalu buat satu koneksi yang memakai token login.

Install dependency

```
npm install socket.io-client
```

src/services/socket.ts

```
import { io } from 'socket.io-client';

const WS_URL = import.meta.env.VITE_WS_URL ||
  'http://localhost:5000';

export const socket = io(WS_URL, {
  autoConnect: false,
  auth: { token: localStorage.getItem('token') }
});
```

Connect dan listen event

```
useEffect(() => {
  if (!token) return;
  socket.auth = { token };
  socket.connect();

  socket.on('receiveMessage', (message) => {
    setMessages((current) => [...current, message]);
  });

  socket.on('messageStatusUpdate', ({ messageId, status }) => {
    setMessages((current) => current.map((message) =>
      message.id === messageId ? { ...message, status } : message
    ));
  });

  return () => {
    socket.removeAllListeners();
    socket.disconnect();
  };
}, [token]);
```

Mengirim pesan

```
socket.emit('sendMessage', {
  chatId: activeChat,
  content: inputValue
});
```

Hapus simulasi

Setelah Socket.IO aktif, hapus setTimeout yang mengubah status pesan secara lokal. Status harus berasal dari event backend.

33. JOIN DAN LEAVE ROOM CHAT**Sinkronisasi room**

```
useEffect(() => {
  if (!activeChat || !socket.connected) return;
  socket.emit('joinChat', { chatId: activeChat });

  return () => {
    socket.emit('leaveChat', { chatId: activeChat });
  };
}, [activeChat]);
```

Backend harus mengabaikan senderId dari payload client dan memakai identitas user hasil verifikasi JWT pada Socket.IO session.

34. MENYELESAIKAN FRIEND REQUEST

Handler accept dan reject pada Chat.tsx masih mengubah state lokal. Aktifkan endpoint backend agar perubahan persisten.

Accept request

```
const result = await api.post(
  `/friend-requests/${requestId}/accept`
);
setFriendRequests((current) =>
  current.filter((request) => request.id !== requestId)
);
setChats((current) => [result.chat, ...current]);
```

Reject request

```
await api.post(`/friend-requests/${requestId}/reject`);
setFriendRequests((current) =>
  current.filter((request) => request.id !== requestId)
);
```

35. STATUS HALAMAN GROUP, FILES, DAN MEMBERS

Halaman	Kondisi Saat Ini	Langkah Integrasi
GroupChats.tsx	Data dan pesan mock	GET /groups dan Socket.IO
Files.tsx	Daftar file mock	GET/POST/DELETE /files
Members.tsx	Direktori member mock	GET /members

Urutan yang disarankan

Stabilkan login, register, chat list, message history, dan Socket.IO lebih dahulu. Setelah itu pindahkan GroupChats, Files, dan Members dari mock data ke backend.

36. PENGUJIAN FULL-STACK LOKAL

23. Jalankan backend pada `http://localhost:5000`.
24. Pastikan CORS backend mengizinkan `http://localhost:3000`.
25. Isi `VITE_API_URL` dan `VITE_WS_URL` pada `.env.local`.
26. Jalankan `npm run dev`.
27. Register atau login memakai akun seed.
28. Periksa Network tab untuk status REST API.
29. Periksa koneksi Socket.IO dan kirim pesan dari dua browser.

Validasi source frontend sebelum deployment

```
npm run lint
npm run build
```

Skenario	Hasil yang Diharapkan
Login valid	Redirect ke /chat dan token tersimpan
Refresh /chat	Session dipulihkan dari localStorage
Token invalid	Session dihapus dan redirect /login
GET /chats	Sidebar menampilkan data backend
Kirim pesan	Browser lain menerima tanpa refresh
Accept request	Direct chat baru muncul

37. DOCKERFILE FRONTEND DAN NGINX**Dockerfile production**

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
ARG VITE_API_URL
ARG VITE_WS_URL
ENV VITE_API_URL=$VITE_API_URL
ENV VITE_WS_URL=$VITE_WS_URL
RUN npm run build

FROM nginx:alpine
RUN rm /etc/nginx/conf.d/default.conf
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

SPA fallback pada nginx.conf

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
}
```

38. BUILD DAN MENJALANKAN IMAGE FRONTEND

```
docker build \
  --build-arg VITE_API_URL=https://api.example.com/api \
  --build-arg VITE_WS_URL=https://api.example.com \
  -t USERNAME/chat-frontend:latest .

docker run -d \
  --name chat-frontend \
  --restart unless-stopped \
  -p 80:80 \
  USERNAME/chat-frontend:latest
```

Build-time configuration

VITE_API_URL harus berisi prefix /api, sedangkan VITE_WS_URL menunjuk origin Socket.IO tanpa suffix /api.

39. DEPLOYMENT FRONTEND KE AWS EC2

30. Buat EC2 Ubuntu dan buka port 22, 80, dan 443.
31. Install Docker pada instance.
32. Pull image frontend dari Docker Hub atau Amazon ECR.
33. Jalankan container frontend pada port 80.
34. Arahkan DNS frontend ke Elastic IP EC2.
35. Pasang HTTPS menggunakan ACM/ALB atau Certbot.

Deployment container

```
docker pull USERNAME/chat-frontend:latest
docker stop chat-frontend || true
docker rm chat-frontend || true
docker run -d \
  --name chat-frontend \
  -p 80:80 \
  --restart always \
  USERNAME/chat-frontend:latest
```

40. DOMAIN, HTTPS, CORS, DAN WSS

Contoh topologi production menggunakan <https://chat.example.com> untuk frontend dan <https://api.example.com> untuk backend. Browser memanggil REST melalui HTTPS dan Socket.IO melalui WSS.

Komponen	Nilai Production
Frontend	https://chat.example.com

VITE_API_URL	https://api.example.com/api
VITE_WS_URL	https://api.example.com
Backend CORS	["https://chat.example.com"]
Protocol realtime	WSS melalui port 443

Mixed content

Frontend HTTPS tidak boleh memanggil API atau Socket.IO melalui HTTP. Gunakan HTTPS/WSS pada kedua sisi.

41. TROUBLESHOOTING FRONTEND

Masalah	Penyebab dan Penyelesaian
Frontend tetap memakai mock	Isi VITE_API_URL lalu restart Vite.
Failed to fetch	Periksa URL, backend, CORS, dan firewall.
401 lalu kembali login	Token invalid/expired; login ulang.
Route 404 setelah refresh	Pastikan try_files ke index.html.
Socket tidak connect	Periksa VITE_WS_URL, token, path, dan WSS.
Pesan tampil dua kali	Hindari listener ganda dan cleanup effect.
URL production tidak berubah	Build ulang image dengan build-arg.
Download file gagal	Gunakan fetch blob, bukan api.get JSON.

42. CHECKLIST INTEGRASI END-TO-END

- Backend health check dan Swagger dapat diakses.
- CORS backend mengizinkan origin frontend.
- Login/register mengembalikan token dan user.id.
- Route privat hanya terbuka setelah autentikasi.
- Chat, member, dan friend request berasal dari backend.
- Message history dimuat saat activeChat berubah.
- Socket.IO mengirim token pada handshake.
- joinChat, leaveChat, sendMessage, dan receiveMessage berfungsi.
- Simulasi status pesan lokal sudah dihapus.
- Image frontend dibangun dengan API/WS URL production.
- Frontend dan backend memakai HTTPS/WSS.
- Secret dan private key tidak tersimpan di Git.

43. KESIMPULAN FULL-STACK

Hands-on backend BECHAT telah dilanjutkan dengan frontend FECHAT. Frontend menggunakan React, Vite, TypeScript, route guard, AuthContext, dan API wrapper untuk mengonsumsi layanan backend secara aman.

Integrasi dasar REST telah memiliki titik implementasi langsung pada Login, Register, dan Chat. Message history, Socket.IO, accept/reject friend request, serta halaman Group, Files, dan Members dijelaskan sebagai tahap implementasi lanjutan berdasarkan kondisi source aktual.

Docker multi-stage membangun bundle Vite dan Nginx menyajikan SPA. Dengan URL API/WS yang benar, CORS terbatas, HTTPS/WSS, image registry, dan AWS EC2/ECS, sistem dapat dijalankan sebagai aplikasi chat full-stack berbasis cloud.

Komponen	Status dalam Repository
React/Vite/Tailwind	Tersedia
Routing dan route guard	Tersedia
AuthContext/localStorage	Tersedia
REST API wrapper	Tersedia
Login/register backend	Titik integrasi tersedia
Initial chat data	Titik integrasi tersedia
Socket.IO client	Perlu diaktifkan
Group/Files/Members API	Masih mock
Docker + Nginx SPA	Tersedia
Deployment EC2	Panduan dan script tersedia

Akhir Hands-On

Dokumen menggabungkan implementasi backend BECHAT dan kelanjutan frontend FECHAT berdasarkan source code repository masing-masing.

REFERENSI

36. Dokumentasi FastAPI - <https://fastapi.tiangolo.com>
37. Dokumentasi Socket.IO - <https://socket.io/docs/v4>
38. Dokumentasi SQLAlchemy - <https://docs.sqlalchemy.org>
39. Dokumentasi React - <https://react.dev>
40. Dokumentasi Vite - <https://vite.dev>
41. Dokumentasi React Router - <https://reactrouter.com>
42. Dokumentasi Tailwind CSS - <https://tailwindcss.com>
43. Dokumentasi Docker - <https://docs.docker.com>
44. Dokumentasi Nginx - <https://nginx.org/en/docs>
45. Dokumentasi Amazon EC2 - <https://docs.aws.amazon.com/ec2>
46. Dokumentasi Amazon ECR - <https://docs.aws.amazon.com/ecr>
47. Repository BECHAT - <https://github.com/diffarydk/BECHAT>
48. Repository FECHAT - <https://github.com/diffarydk/chatFE>